

Introduction aux Frameworks et React.js

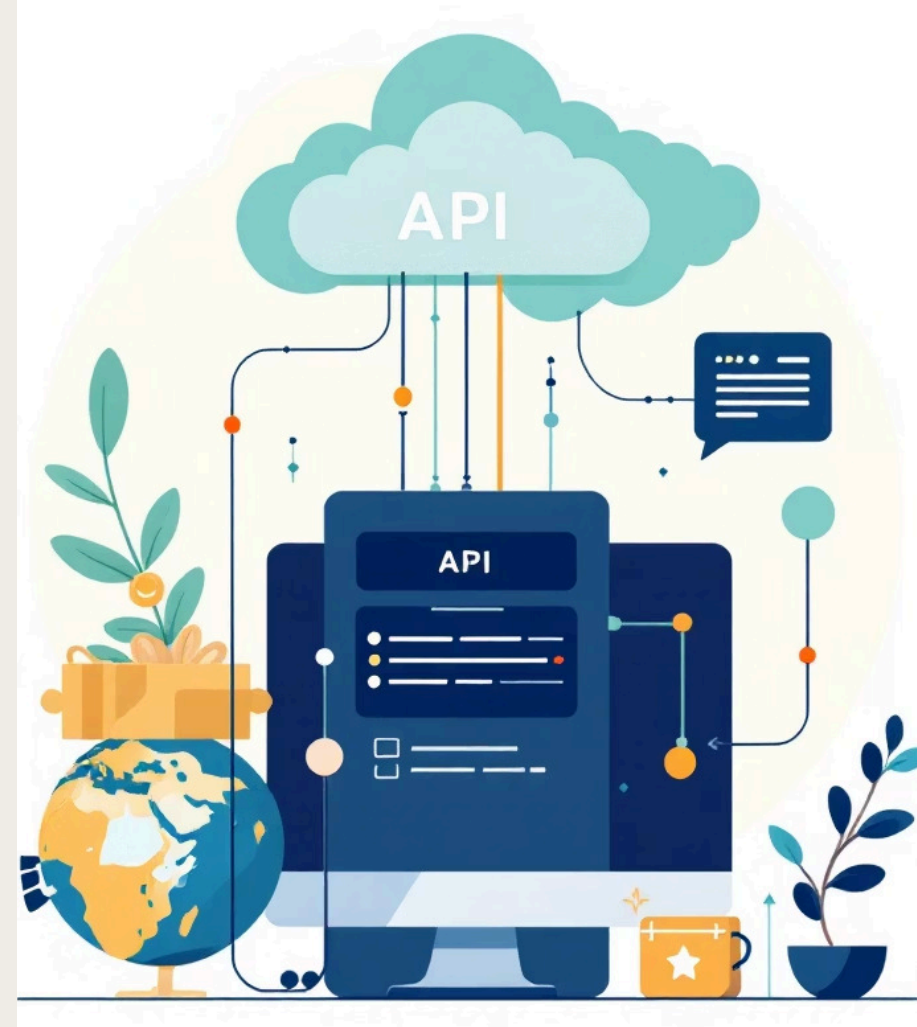
Présentation concise et pratique pour développeurs francophones — style technologique, minimaliste et élégant. Durée estimée : 8–10 minutes.



Le développement web moderne – en quelques mots

- Applications réactives côté client et API côté serveur.
- Modularité, performance et expérience utilisateur au centre.
- Outils : gestionnaires de paquets, bundlers, tests et CI/CD.

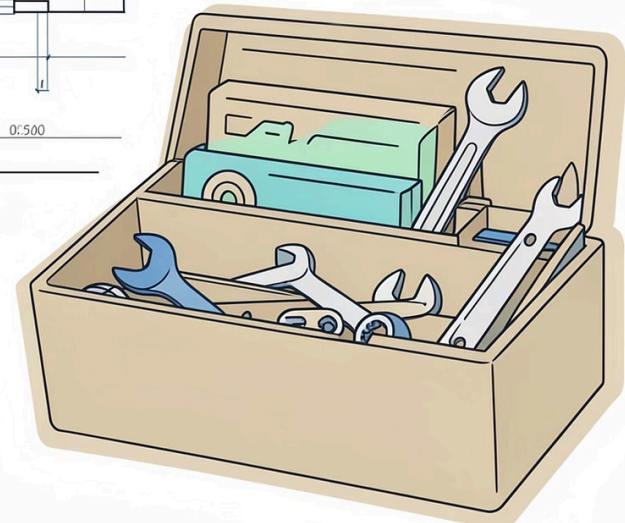
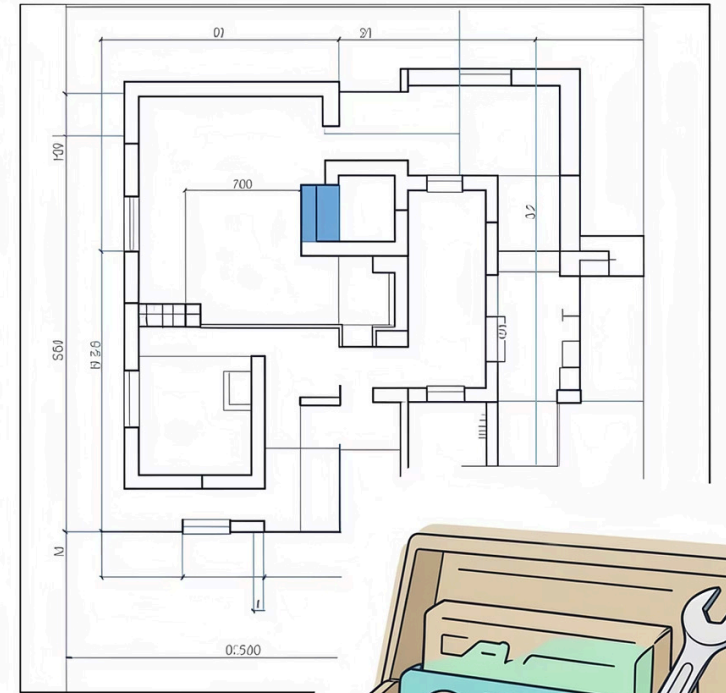
Objectif : accélérer la livraison tout en gardant maintenabilité et performance.

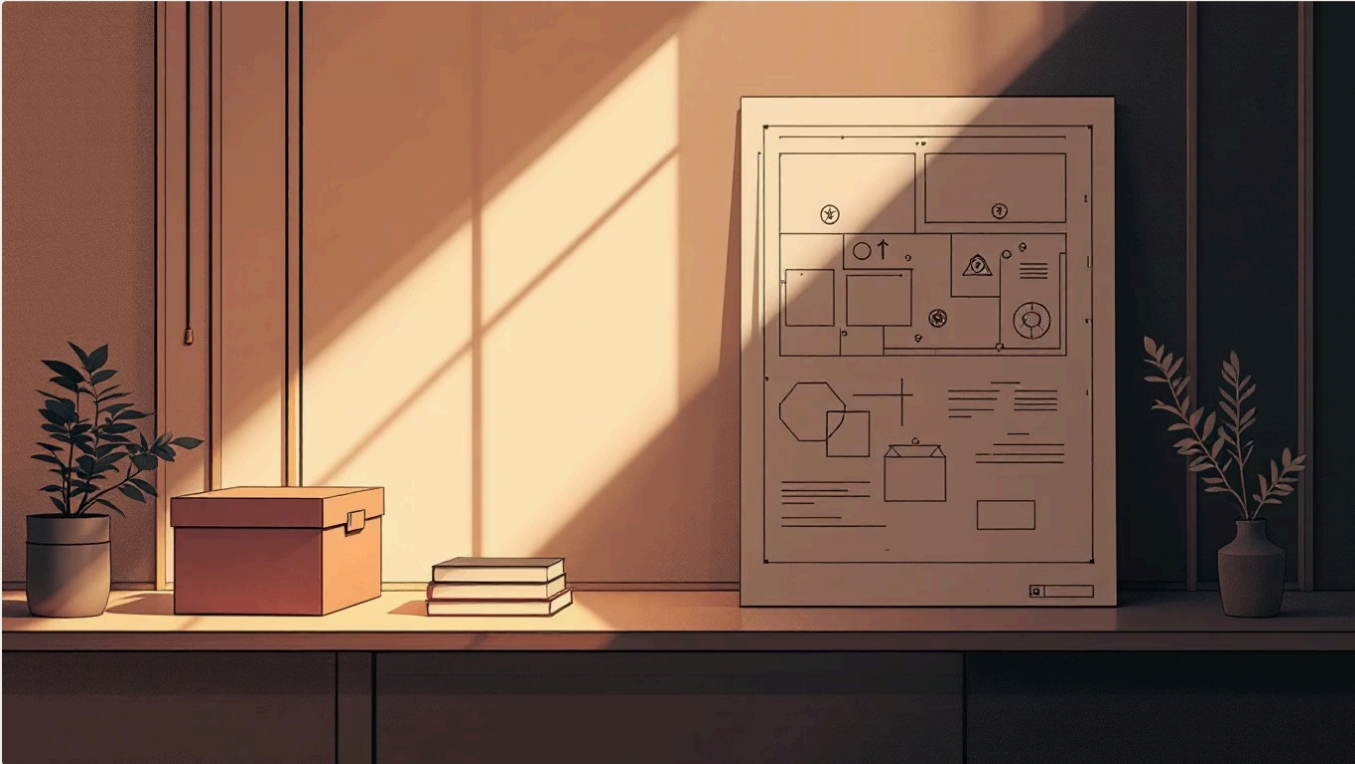


Qu'est-ce qu'un framework ?

Un framework fournit une structure complète et des règles pour construire une application : conventions, cycle de vie, et composants intégrés. Il dicte l'architecture et guide le flux de données.

❏ Exemple d'usage : structurer une application SPA ou une API REST.





Framework vs Library

- **Framework** : inversion de contrôle — il appelle votre code.
- **Library** : collection d'outils que vous appelez quand nécessaire.
- Choix selon : contrôle, flexibilité, courbe d'apprentissage.

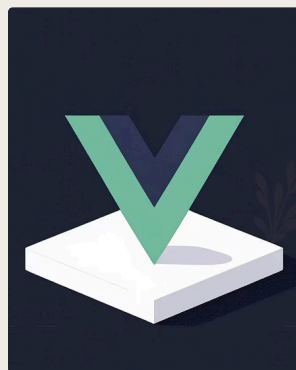
Règle pratique : si l'outil impose l'architecture → framework ; sinon → library.

Exemples de frameworks populaires



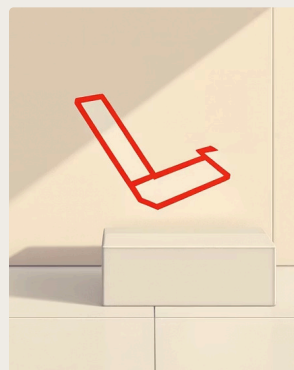
Angular

Framework complet pour applications d'entreprise (TypeScript, convention forte).



Vue.js

Progressive, facile à intégrer, bonne courbe pour prototypes et SPAs.



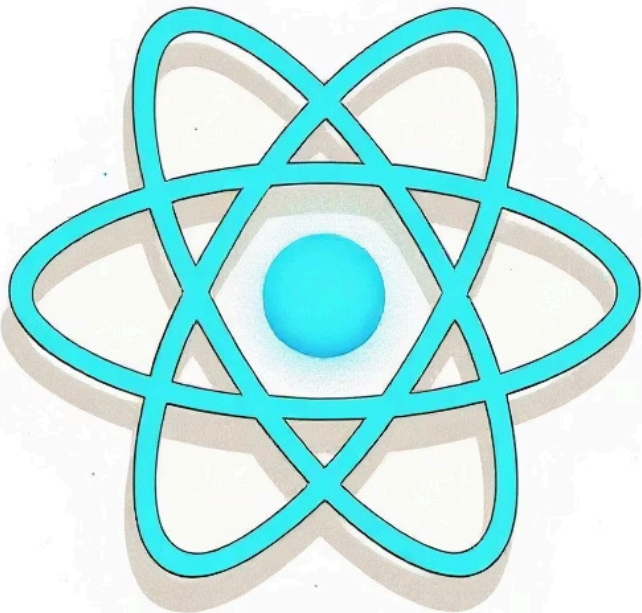
Laravel

Framework PHP moderne pour backends, productivité élevée et écosystème riche.



Django

Framework Python axé sur la rapidité de développement et la sécurité.



React

Présentation de React

Bibliothèque JavaScript pour construire des interfaces utilisateur basées sur des composants réutilisables. Principal atout : modèle déclaratif et arbre d'UI mis à jour efficacement via le virtual DOM.

- ❏ React = library (vous contrôlez l'architecture). Écosystème riche : React Router, Redux/Context, outils de build.



Le concept de composants

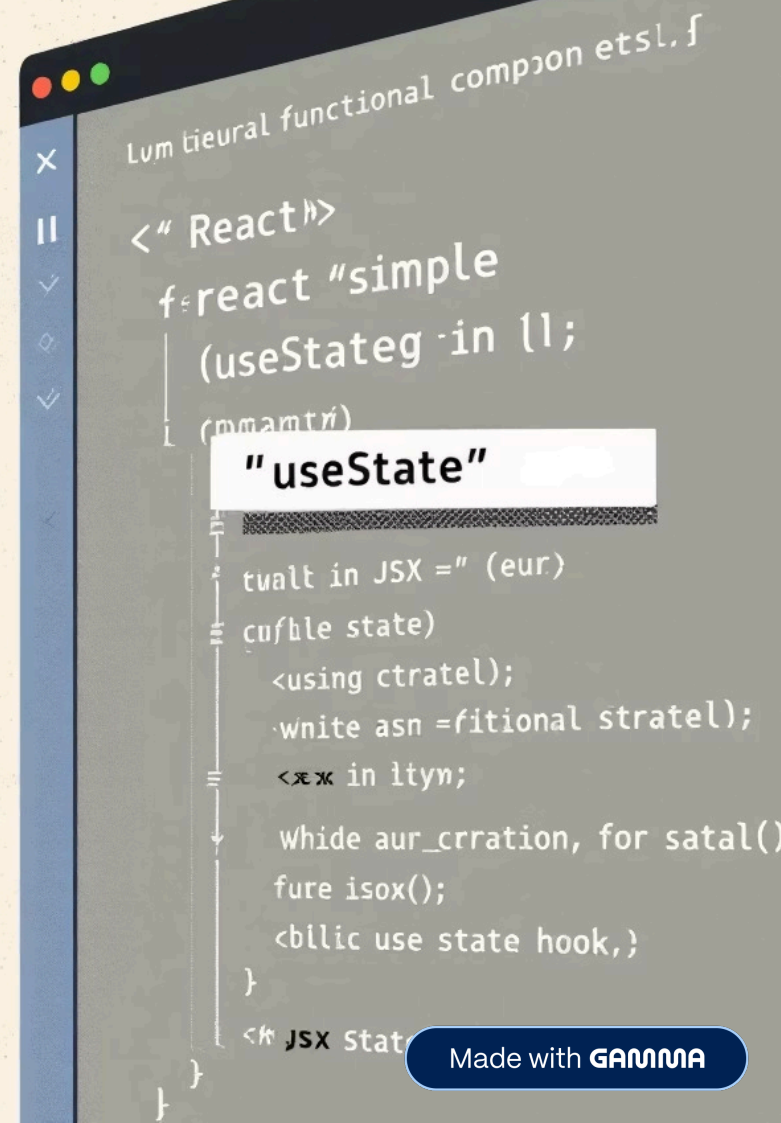
- Bloc d'UI autonome : structure (JSX), style, comportement.
- Props pour configurer, state pour gérer l'état interne.
- Composition > héritage : assembler des composants simples.

Exemple simple de composant React

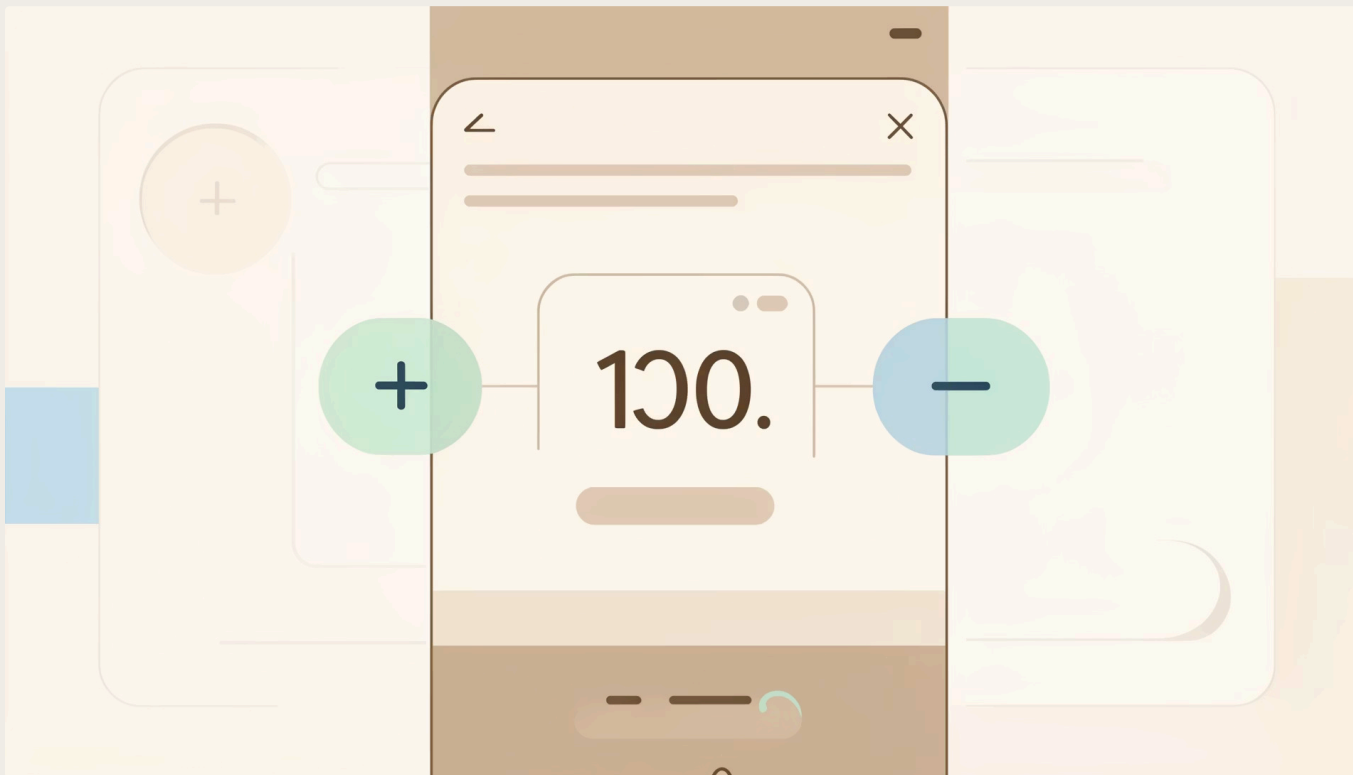
Composant fonctionnel minimal avec état local — exemple prêt à copier :

```
function Greeting({ name }) {  
  return <h1>Bonjour, {name}!</h1>  
}
```

Utilisation : <Greeting name="Amina" />



Exemple : compteur avec useState



```
import React, { useState } from
'react';

function Counter() {
  const [count, setCount] =
  useState(0);
  return (
    <div>
      <h3>Compteur : {count}</h3>
      <button onClick={() =>
setCount(count - 1)}>-</button>
      <button onClick={() =>
setCount(count + 1)}>+</button>
    </div>
  );
}
export default Counter;
```

Points clés : état local, mise à jour
déclarative, simple à tester.

Avantages et conclusion

Productivité

Composants réutilisables et écosystème riche accélèrent le développement.

Performance

Virtual DOM et mises à jour ciblées réduisent les rendus inutiles.

Évolutivité

Architecture par composants facilite la maintenance à long terme.

Prochaines étapes recommandées : créer un petit projet React, explorer Router et la gestion d'état (Context / Redux), et ajouter des tests simples.

